



电子科技大学

University of Electronic Science and Technology of China

信息与软件工程学院

SCHOOL OF INFORMATION AND SOFTWARE ENGINEERING

W2D4/D5: 面向网络的并发编程2 面向网络的并发编程3

Linux下HTTP Server的设计

实训任务概览

W2D4: SelectServer + TCP Client

-> 3 客户端收发、quit 清理、FD_SET/FD_CLR 正确

W2D5: EpollServer 检查点

-> 使用 LT + EPOLLIN 验证多客户端收发

W2D5 最终提交: V1.0 epoll WebServer

-> serve-epoll <max_requests>、路由、404、关闭清理、测试脚本

- 训练目的
- 训练原理
- 训练内容

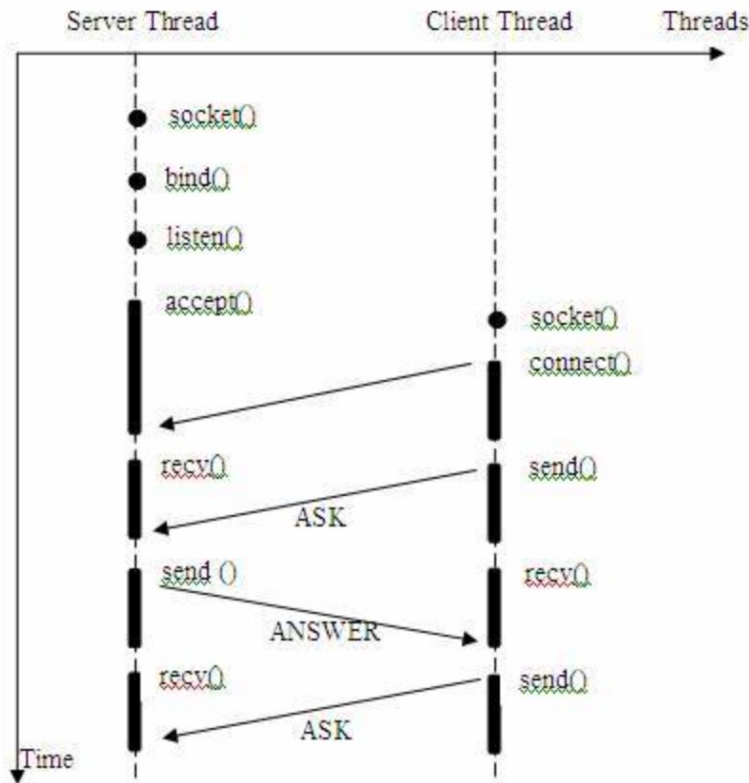


- 熟悉Linux网络编程的不同I/O模式
- 熟悉I/O多路复用的模型和相关函数
- 掌握select服务器编程模型
- 掌握epoll服务器编程模型

简单服务器

多个CLIENT需要通过排

——解决需求



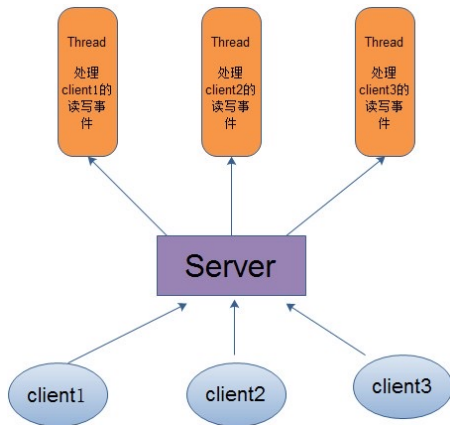
多线程（进程）的并发服务器

对于多线程（进程）模式，以多线程为例：主线程持续等待客户端的连接请求，如果有连接，则创建新线程，并在新线程中提供为前例同样的问答服务。

模式：出现客户端请求，服务器就会新建一个线程来处理该客户端的读写事件。

问题：

这种模式虽然处理起来简单方便，但是由于服务器为每个client的连接都采用一个线程去处理，使得资源占用非常大。因此，当连接数量达到上限时，再有用户请求连接，直接会导致资源瓶颈，严重的可能会直接导致服务器崩溃。



多线程的优化考虑

• 线程池

创建一个固定大小的线程池，来一个客户端，就从线程池取一个空闲线程来处理，当客户端处理完读写操作之后，就交出对线程的占用

• 线程池比较适合大量的短连接应用

“池”始终有其上限，当请求大大超过上限时，效果并不显著。所以“使用“池” 必须考虑其面临的响应规模，并根据响应规模调整“池”的大小。

- 服务器端在实际的工作中需要接受成千上万的并发连接。
- 多线程模型可以方便高效的解决小规模的服务请求，但面对大规模的服务请求，多线程模型并不是最佳方案

因此服务器处理的考虑演变围绕着一个核心需求——**如何用尽可能少的线程去处理更多的连接**

尽可能少的线程（极端=单线程），例如单线程去处理并发客户端请求连接，思考一种方式：

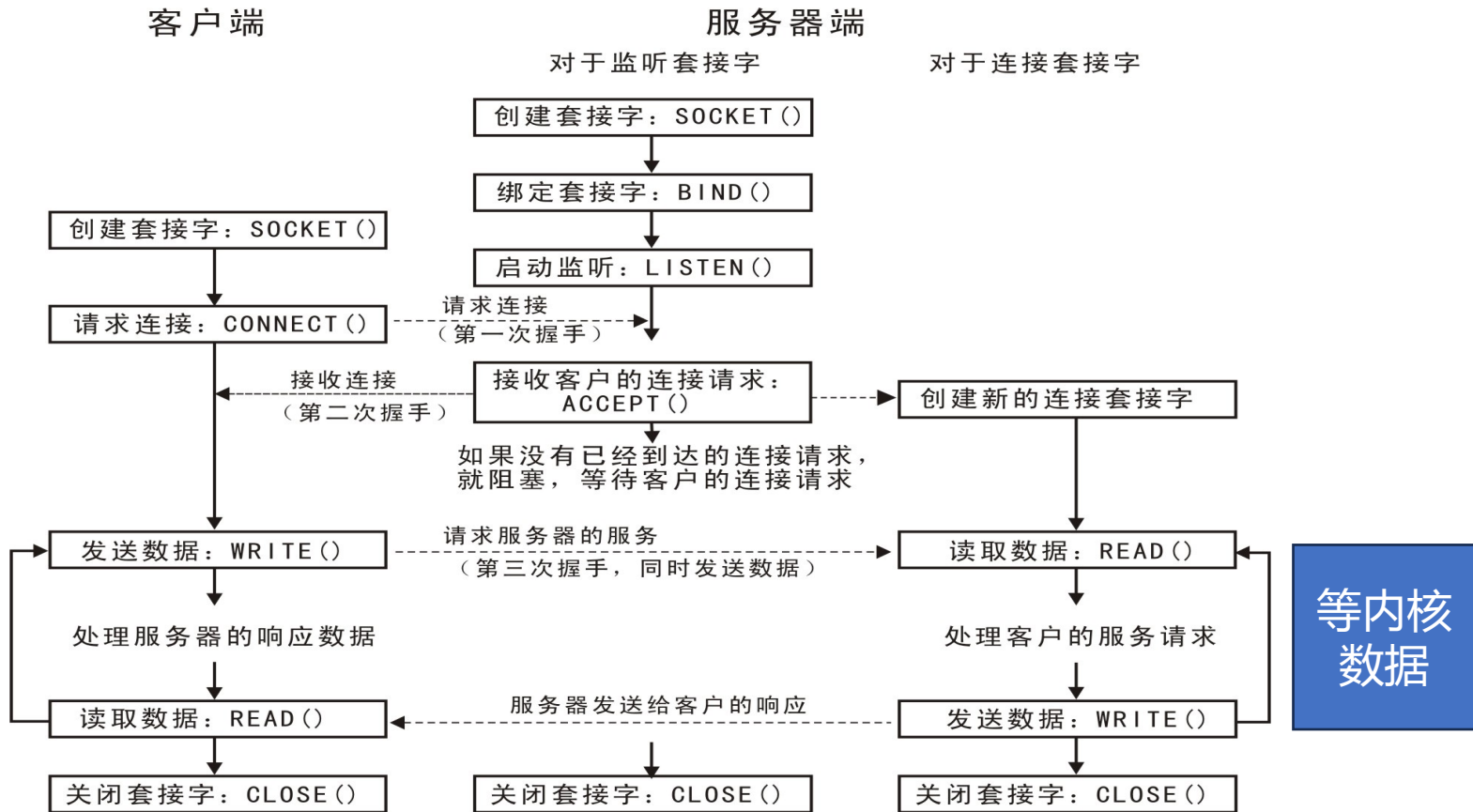
让等待I/O的线程可以分身去处理其他客户端请求。

怎样让线程可以分身处理多个请求？

首先要了解的是**I/O在套接字上工作**的流程，这包括两步：

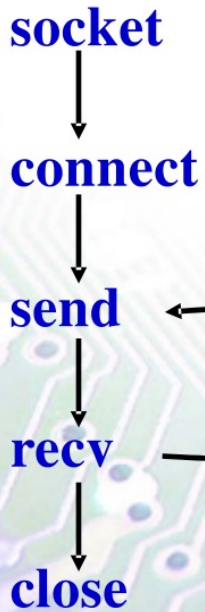
- ① 等待数据进入内核接收缓冲区
- ② 再从内核缓冲区复制到用户缓冲区

这也就告诉我们，IO操作可以分为两个阶段，第一阶段即从应用进程到内核，第二阶段是内核内部的操作。



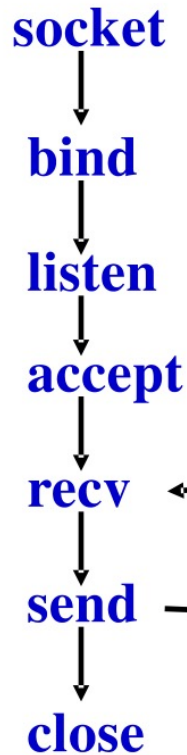
客户端

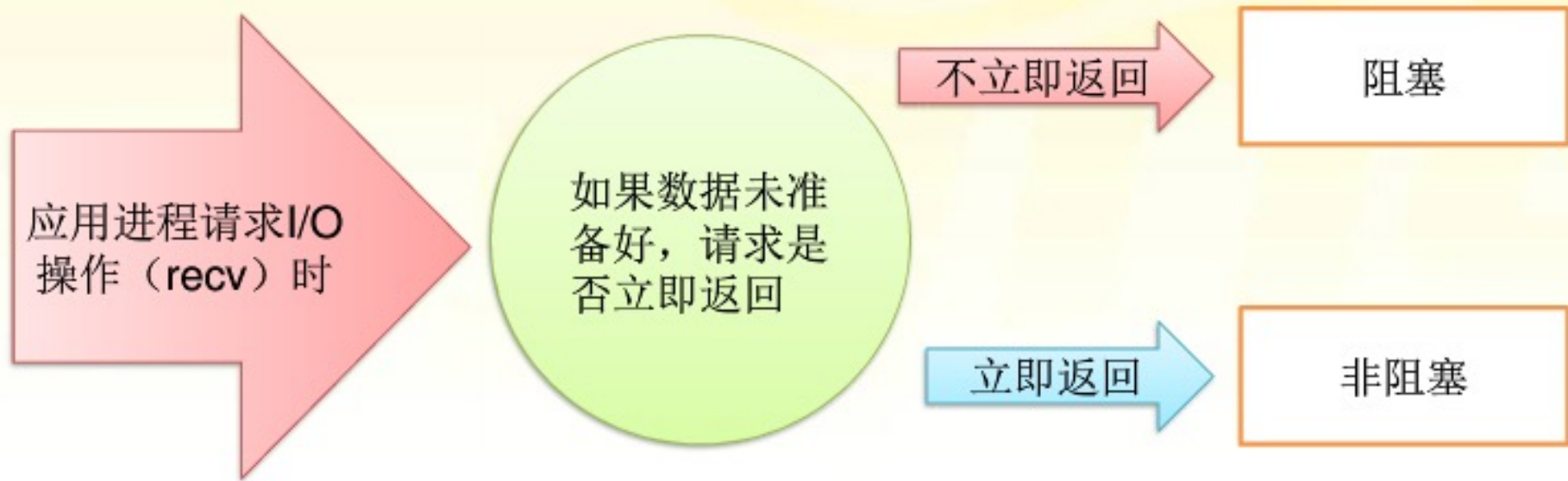
socket
↓
connect
↓
send
↓
recv
↓
close



服务器端

socket
↓
bind
↓
listen
↓
accept
↓
recv
↓
send
↓
close

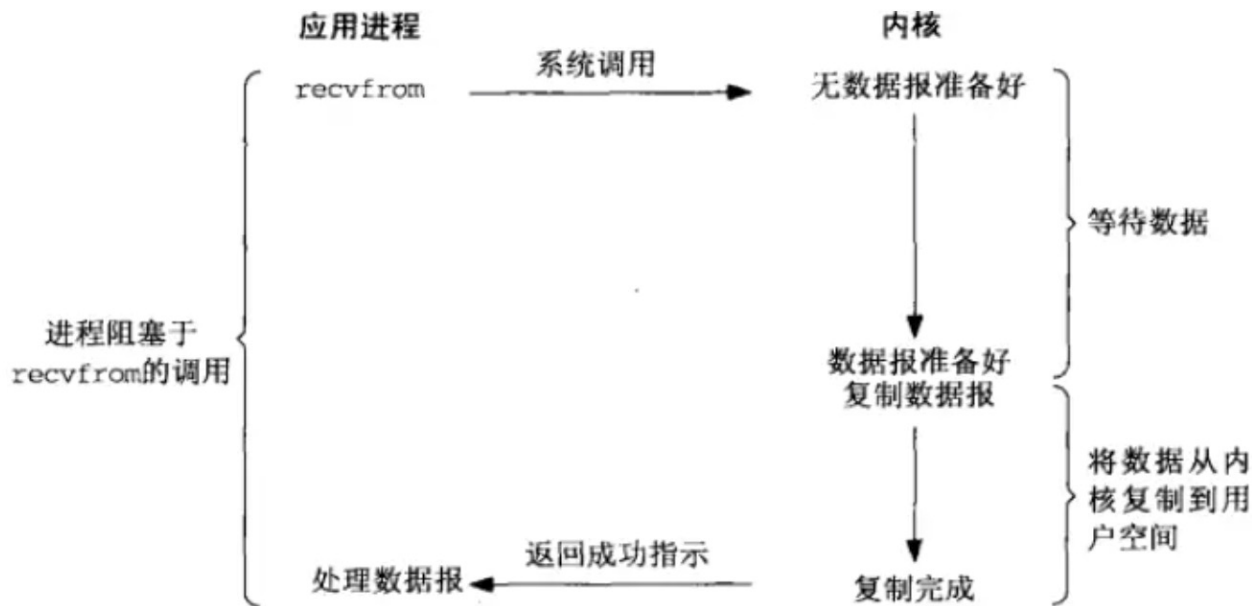




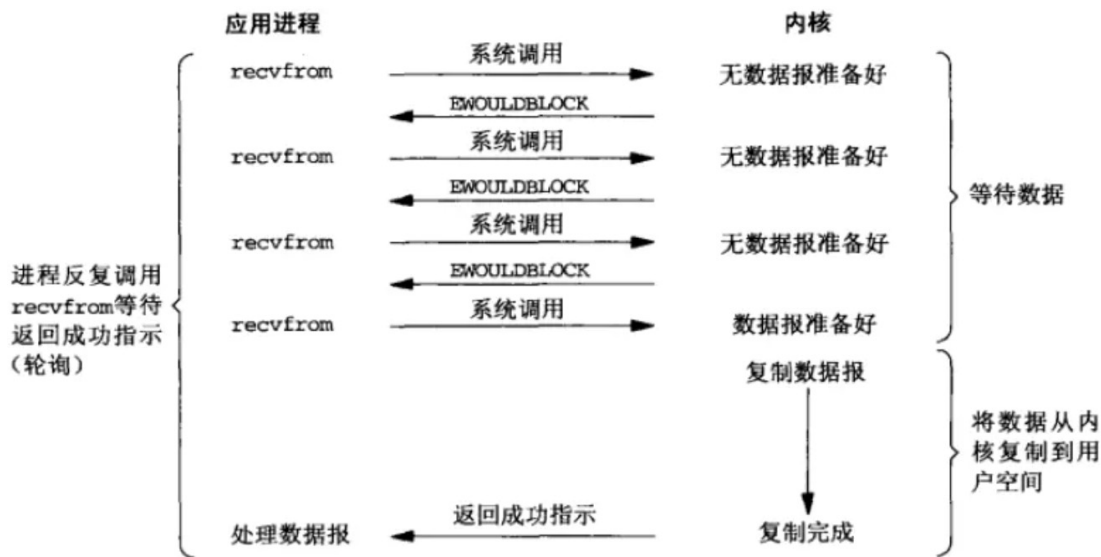
五种I/O模式

- 阻塞IO
- 非阻塞IO
- I/O多路复用
- 信号驱动IO（暂不介绍）
- 异步IO（暂不介绍）

- 阻塞IO: 在内核将数据准备好之前，系统调用会一直等待所有的套接字，默认的是阻塞方式。
- 例子：在write完成之后read才能执行，否则被阻塞；



非阻塞I/O：每次应用询问内核是否有数据准备好，即文件描述符缓冲区是否就绪。当有数据报准备好时，就进行拷贝数据报的操作。当没有数据报准备好时，也不阻塞程序，内核直接返回未准备就绪的信号，等待用户程序的下一个轮询。



I/O多路复用模型

什么是多路？

- 多路：核心需求是要用尽可能少的线程来处理尽可能多的连接，这里的多路指的就是需要处理的众多连接。

什么是复用？

- 复用：核心需求要求使用尽可能少的线程，尽可能少的系统开销去处理尽可能多的连接(多路)。到了I/O多路复用模型中，多个连接可以复用这一个独立的线程去处理多个连接上的读写。

如何去实现这个复用，也就是如何让一个独立的线程去处理众多连接上的读写事件？

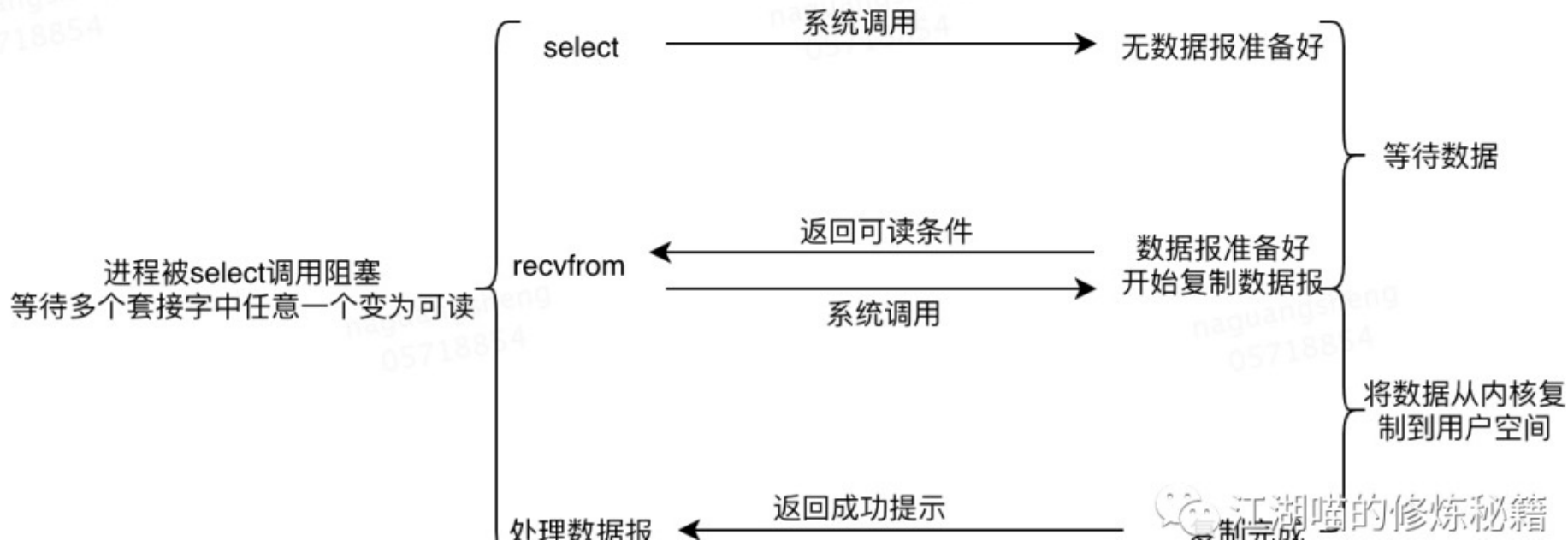
- 在非阻塞I/O模型中，利用非阻塞的系统I/O调用去不断的轮询众多连接的Socket接收缓冲区看是否有数据到来，如果有则处理，如果没有则继续轮询下一个Socket。这样就达到了用一个线程去处理众多连接上的读写事件了。

问题：不断发起系统调用去轮询各个Socket中的接收缓冲区是否有数据到来，频繁系统调用带来了大量的上下文切换开销。随着并发量的提升，这样会导致非常严重的性能问题

- 把频繁的轮询操作交给操作系统内核来完成，避免了在用户空间频繁的去使用系统调用来轮询所带来的性能开销——引出I/O多路复用模型

- I/O多路复用是计算机网络编程中的重要技术，原理是多个网络I/O流通过一个轮询机制集中管理，使程序同时监视多个网络流并且在任何一个网络流产生数据时能够对其进行读取和处理。
- 在Unix/Linux操作系统中，I/O多路复用常常使用select()、poll()以及epoll()等函数来实现。

以select为例的I/O多路复用



I/O 多路技术一般在下面这些情况中使用：

- 需要同时处理多个文件描述符的输入输出操作的时候（一般来说是标准的输入输出和网络套接字）
- 程序需要同时进行多个套接字的操作的时候。
- 一个TCP服务器程序同时处理正在侦听网络连接的套接字和已经连接好的套接字。
- 如果一个服务器程序同时使用 TCP和UDP 协议。
- 如果一个服务器同时使用多种服务并且每种服务可能使用不同的协议（比如inetd就是这样）

I/O多路复用的三种函数

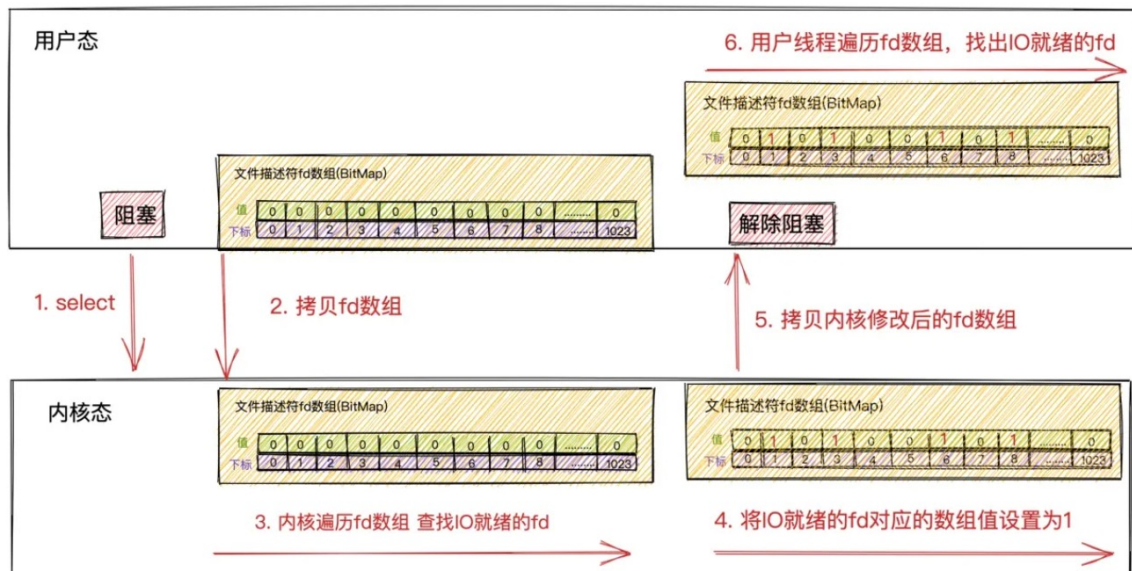
- Select函数模型
- Poll函数模型
- Epoll函数模型

简介：select诞生于4.2BSD，在几乎所有平台上都支持，具有良好的跨平台性。Linux提供select函数来实现多路复用输入/输出模型。

- Select函数在一组文件描述符 (普通文件, 设备文件-终端, socket, pipe, ...) 上使用 blocking I/O。
- 可以请求select在数据到来时通知去读取数据，不管数据来自 STDIN还是TCP/UDP socket或者其他文件。

Select模型工作原理

select系统调用将轮询的操作交给了内核来帮助我们完成，从而避免了在用户空间不断的发起轮询所带来的的系统性能开销



监听Socket数组（即文件描述符数组）：

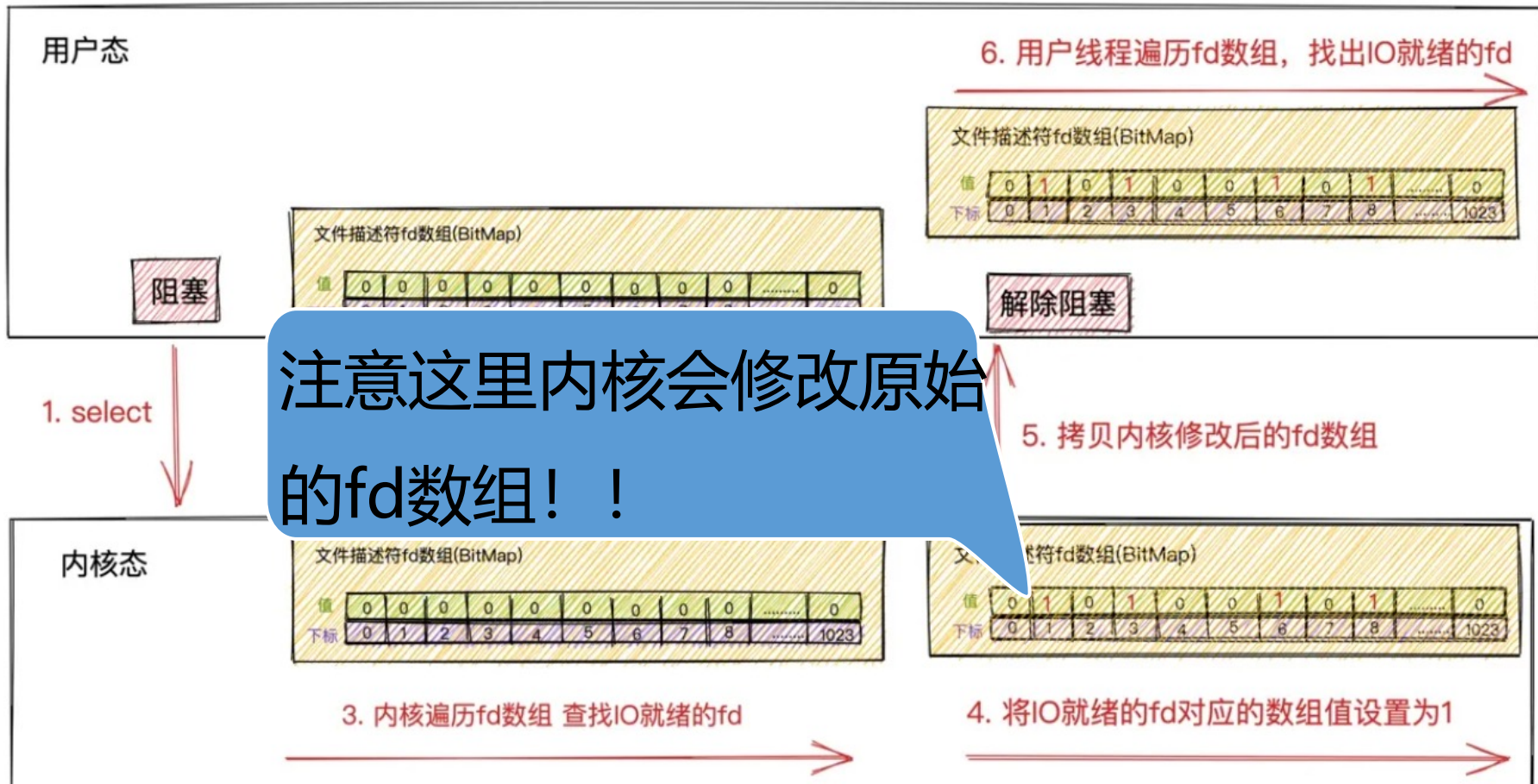
这里的文件描述符数组其实是一个BitMap

BitMap下标为文件描述符fd，对应的值为：

- ✓ 1表示该fd上有读写事件；
- ✓ 0表示该fd上没有读写事件。

文件描述符fd数组(BitMap)

值	0	0	0	0	0	0	0		0
fd	0	1	2	3	4	5	6	7	 1023



- ① 内核遍历一遍fd数组后，如果发现有些fd上有I/O数据到来，则将修改后的fd数组返回给用户线程。此时，会将fd数组从内核空间拷贝到用户空间
- ② 当内核将修改后的fd数组返回给用户线程后，用户线程解除阻塞，由用户线程开始遍历fd数组然后找出fd数组中值为1的Socket文件描述符。最后对这些Socket发起系统调用读取数据。

`select`不会告诉用户线程具体哪些fd上有I/O数据到来，只是在I/O活跃的fd上打上标记，将打好标记的完整fd数组返回给用户线程，所以用户线程还需要遍历fd数组找出具体哪些fd上有I/O数据到来。

- ① 在用户线程遍历完fd数组后获取到I/O就绪的Socket 后，需要重置fd数组，并重新调用select 传入重置后的fd数组，让内核发起新一轮遍历轮询。

API介绍

```
int select(int nfd, fd_set *readfds, fd_set *writefds, fd_set *exceptfds, struct timeval *timeout);
```

- **nfd**: 监视的最大文件描述符（即最大有效的文件描述符+1）。
- **readfds**: 待监视的网络I/O流集合，用于检查是否存在可读事件（即是否存在待读取数据）。

这里的fd_set就是我们前边提到的文件描述符数组，是一个BitMap 结构。

注意：

- select的第一个参数为所有监视的**文件描述符的最大值+1**，而不是监视的文件描述符个数+1。在linux下是值而不是个数。
- Linux下文件描述符最大FD_SETSIZE=1024，即对描述符值是0~1023的句柄进行检测。 FD_ISSET 是宏，不会返回负值，fd 越界属于未定义行为。

```
int select(int nfd, fd_set *readfds, fd_set *writefds, fd_set  
*exceptfds, struct timeval *timeout);
```

- **writefds:** 待监视的网络I/O流集合，用于检查是否存在可写事件（即是否存在待写入数据）。
- **exceptfds:** 待监视的网络I/O流集合，用于检查是否存在异常事件。
- **timeout:** 最大等待时间，当指定的时间通过后如果仍然没有事件发生，则select()函数将退出

```
struct timeval {  
    long tv_sec;    /* seconds */  
    long tv_usec;   /* microseconds */  
}  
struct timeval max = {1,0};
```

struct timeval *timeout是最大等待时间，也是 select的超时时间，这个参数很重要，它可以使 select 处于三种状态。

- timeout=NULL，一直等待 — 直到有描述符可以I/O返回
- timeout设置具体值，等待一定的时间 — 有描述符可以I/O；或者时间到返回.
- timeout = {0,0}，不等待— 检查了文件描述符后立即返回. 这样相当于一个 “轮询”

```
int select(int nfd, fd_set *readfds, fd_set *writefds, fd_set  
*exceptfds, struct timeval *timeout);
```

- 成功：返回描述符集中有多少个 “可用” ；
- 如果select() 超时，返回0；
- 出错返回-1，并设置错误码
 - EBADF 有一个错误的描述符.
 - EINTR 收到一个信号.
 - EINVAL 参数错误.
 - ENOMEM 内存不足.

在用户线程中重新遍历fd数组的过程中，需要用到的API：

- **void FD_ZERO(fd_set *fdset):** 清空指定的文件描述符集合，即让fd_set 中不再包含任何文件描述符。
- **void FD_SET(int fd, fd_set *fdset):** 将一个给定的文件描述符加入集合之中。

每次调用select之前都要通过FD_ZERO和FD_SET重新设置文件描述符，因为文件描述符集合会在内核中被修改。

在用户线程中重新遍历fd数组的过程中，需要用到的API：

- **int FD_ISSET(int fd, fd_set *fdset):** 检查集合中指定的文件描述符是否可以读写。
 - 1. 当描述符存在于指定的集合中时，返回真；
 - 2. 当描述符不存在于指定的集合中时，返回假；
 - 3. 当参数错误时，或者文件描述符超出了指定集合中的最大值，则返回一个负值。
- **void FD_CLR(int fd, fd_set *fdset):** 将一个给定的文件描述符从集合中删除

■ 习题：

fd_set 数组结构

fd0 fd1 fd2 fd3 fd4 fd5 fd6 fd7

调用 FD_ZERO (&set)

0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---

fd=5 fd=2 fd=1

调用 FD_SET (fd, &set)

0	1	1	0	0	1	0	0
---	---	---	---	---	---	---	---

执行 select (6, &set, 0, 0, 0) 阻塞等待

若 fd=1, fd=2, 上都发生可读事件 则, select 返回

0	1	1	0	0	0	0	0
---	---	---	---	---	---	---	---

- 习题

Select 函数的readfds 参数传入时和参数传出时

入参时 *readfds

0	0	0	0	0	1	0	1
---	---	---	---	---	---	---	---

监听哪些FD

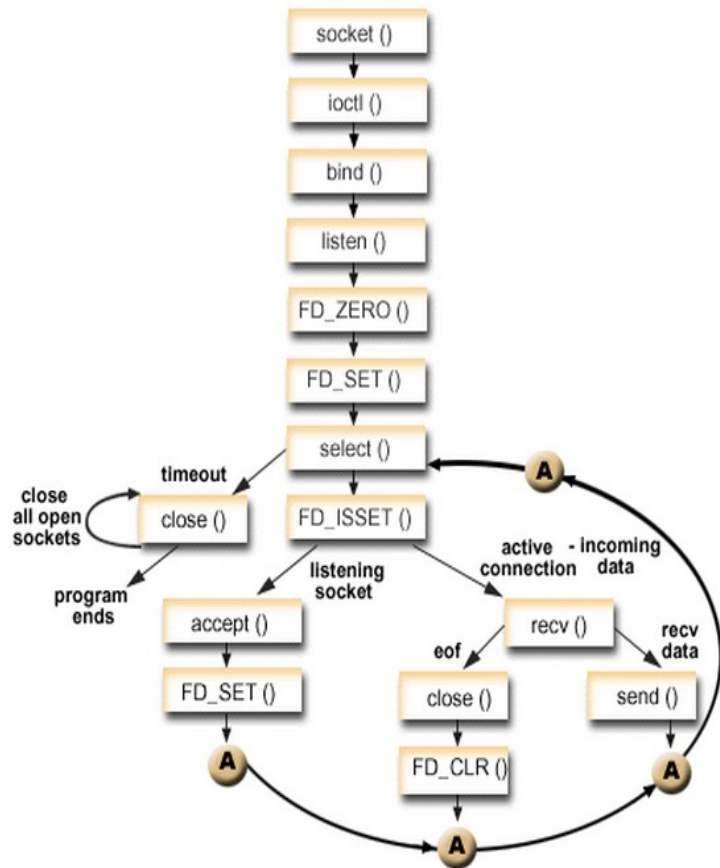
回参时 *readfds

0	0	0	0	0	1	0	0
---	---	---	---	---	---	---	---

哪些FD就绪了

- ① 创建 fd_set xxxset
- ② 采用 FD_ZERO 清空 xxxset
- ③ 逐个将每个需监视的描述符添加到 xxxset中.
- ④ 调用 select()
- ⑤ 一旦 select() 返回, 使用 FD_ISSET 确认所需要的描述符上的 I/O 是否能够进行操作 (该位是否1)
- ⑥ 根据不同的描述符进行不同的操作。

训练原理：select模型_伪代码



- Linux下单个进程能够监视的文件描述符的数量存在最大限制，通常是1024，可以更改数量，但由于select采用轮询的方式扫描文件描述符，文件描述符数量越多，性能越差；
(在linux内核头文件中，有这样的定义:#define __FD_SETSIZE 1024)
- 内核/用户空间内存拷贝问题，select需要复制大量的句柄数据结构，产生巨大的开销；
- select返回的是含有整个句柄的数组，应用程序需要遍历整个数组才能发现哪些句柄发生了事件；
- 应用程序如果没有完成对一个已经就绪的文件描述符进行I/O操作，那么之后每次select调用还是会将这些文件描述符通知进程。

Poll函数

poll相当于是改进版的select，但是工作原理基本和select 没有本质的区别

```
int poll(struct pollfd *fds, unsigned int nfds, int timeout)
struct pollfd{
int fd; /* 文件描述符 */
short events; /* 需要监听的事件 */
short revents; /* 实际发生的事件 由内核修改设置 */};
```

- Select: 固定长度为1024的BitMap结构的fd_set，
- Poll: pollfd结构，没有固定长度的数组，没有最大描述符数量的限制(但是还会受到系统文件描述符限制)

简介：epoll诞生于Linux 2.6内核，被公认为是Linux2.6下性能最好的多路I/O复用方法。epoll是Linux内核为处理大批量句柄而作了改进的poll，是Linux下多路复用I/O接口select/poll的增强版本，它能显著减少程序在大量并发连接中只有少量活跃的情况下的系统CPU利用率。

- 支持一个进程打开大数目的socket描述符
- I/O效率不随fd数目增加而线性下降
- 实现高效处理百万文件描述符

出现背景

有100万个客户端同时与一个服务器进程保持着TCP连接。每一时刻，通常只有几百上千个TCP连接是活跃的(事实上大部分场景都是这种情况)。如何实现这样的高并发？

在select/poll时代，服务器进程每次都把这100万个连接告诉操作系统(从用户态复制句柄数据结构到内核态)，让操作系统内核去查询这些套接字上是否有事件发生，轮询完后，再将句柄数据复制到用户态，让服务器应用程序轮询处理已发生的网络事件，这一过程资源消耗较大，因此，select/poll一般只能处理几千的并发连接。

`poll()` 和 `epoll` 都属于 I/O 多路复用技术：用一个线程同时等待多个 `socket` 的连接、数据和断开事件，避免阻塞在某一个客户端上

o

主要区别

对比项	<code>poll()</code>	<code>epoll</code>
使用范围	POSIX, Linux、Unix 等系统可用	Linux 专用
fd 管理方式	每次调用都传入完整 <code>pollfd</code> 数组	使用 <code>epoll_ctl()</code> 提前注册 fd
等待函数	<code>poll()</code>	<code>epoll_wait()</code>
查找就绪 fd	返回后通常遍历整个数组	直接返回就绪事件数组
数据传递	每轮把完整 fd 数组交给内核	fd 注册一次，后续主要获取就绪事件
大量连接性能	fd 较多时遍历成本明显	更适合大量连接、少量活跃的场景
触发模式	主要是水平触发	支持 LT 水平触发和 ET 边沿触发
编程复杂度	较简单	稍复杂，需要维护注册、修改和删除
fd 数量限制	没有 <code>select</code> 的 <code>FD_SETSIZE</code> 固定限制	同样没有 <code>FD_SETSIZE</code> 固定限制

连接数量较少时，`poll` 和 `epoll` 的实际性能差异可能很小。`epoll` 的优势主要体现在“连接很多，但每次只有少量连接活跃”的服务器场景。

select

- -> 使用 fd_set
- -> 每轮复制集合
- -> 存在 FD_SETSIZE 限制

poll

- -> 使用 pollfd 数组
- -> 没有 FD_SETSIZE 固定限制
- -> 每轮仍需提交并遍历完整数组

epoll

- -> fd 提前注册到内核
- -> epoll_wait 直接返回就绪事件
- -> 更适合大量长连接

poll 解决了 select 的 FD_SETSIZE 固定限制，但仍需每轮提交并遍历完整 fd 数组。本次课程直接从 select 过渡到 Linux 更常用的 epoll，因此 poll 只作为对比知识，不作为实训成果。

epoll的调用分成了3个部分：

- ① 调用**epoll_create1()**建立一个epoll对象(在epoll文件系统中为这个句柄对象分配资源)，得到 `epoll_fd`
- ② 调用**epoll_ctl()**向 `epoll_fd` 添加、修改或删除监听对象
- ③ 调用**epoll_wait()**收集发生的事件的连接，等待事件，返回已经就绪的 `fd`

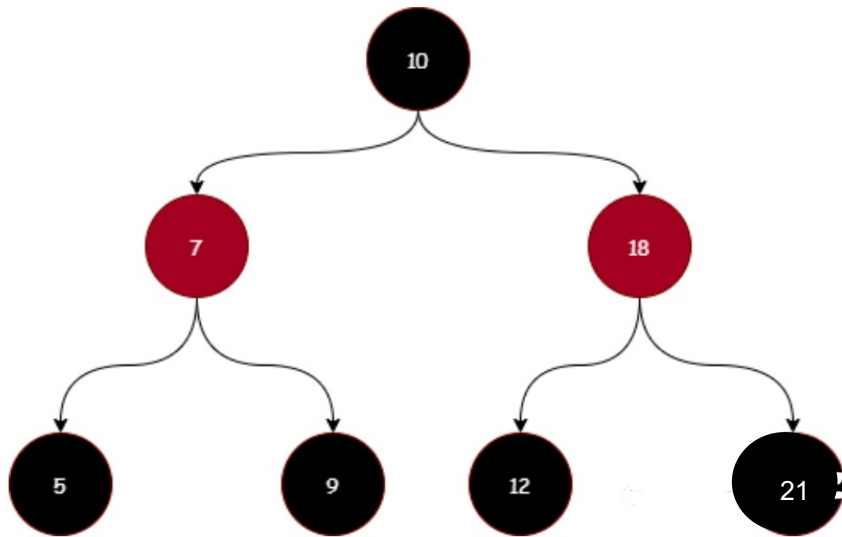
```
int epoll_create(int size);  
int epoll_create1(int flags);
```

两个函数都可以创建 **epoll** 实例，但现代 Linux 程序建议使用 `epoll_create1()`。

在上述的三个部分中，有两个重要的数据结构提供了帮助，以至于epoll 在性能上有较大的提升。

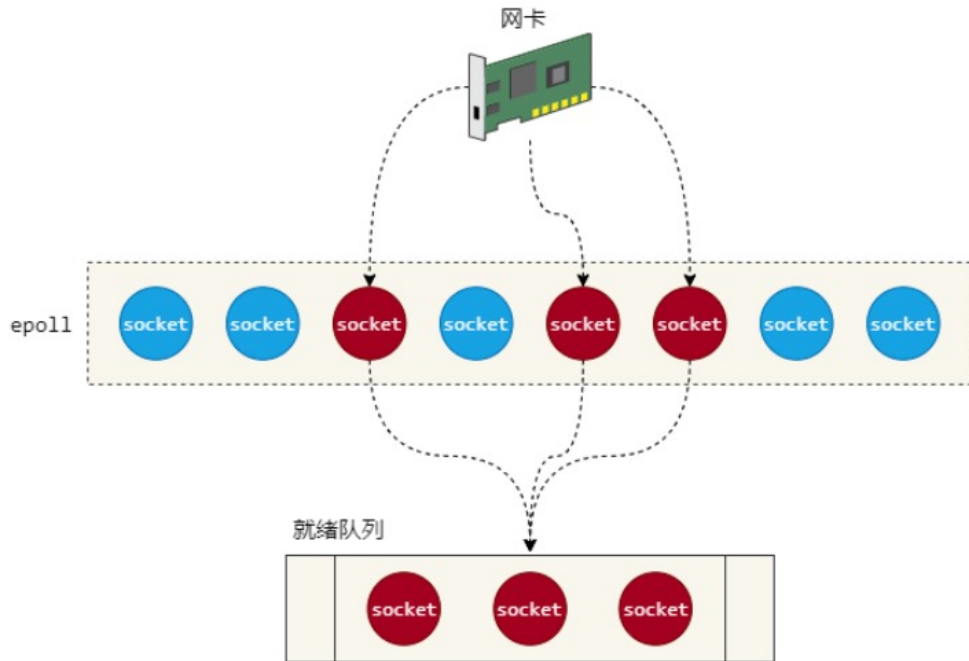
重要数据结构——红黑树

Epoll内部使用红黑树保存所有监听的 socket，红黑树是一种平衡二叉树，添加和查找元素的时间复杂度为 $O(\log n)$ ，每个节点中的数字代表着 socket 句柄



重要数据结构——就绪队列

epoll 还维护着一个就绪队列，当epoll监听的socket状态发生改变（变为可读或可写）时，就会把就绪的socket 添加到就绪队列中



- 当socket从网络中获取到数据后，会发通知给 epoll，epoll 会将当前 socket 添加到就绪队列中，并且唤醒等待中的进程（调用 epoll_wait的进程）。
- 当 socket 状态发生变化时，会调用 ep_poll_callback 函数来通知 epoll。该函数的主要工作为：
 - ✓ 把就绪的socket添加到就绪队列中。
 - ✓ 唤醒调用epoll_wait函数而被阻塞的进程。
 - ✓ 当进程被唤醒后，就会从就绪队列中把就绪的socket复制到用户提供的数组中。

用户怎么操作**就绪的描述符**？

- 在调用epoll_wait时用户程序需要提供一个events数组，用于存储就绪的socket。
- 当epoll_wait返回后，用户就可以从events数组中获取到就绪的socket，并可对其进行读写操作。

EPOLL三个函数

- **epoll_create1()** 系统调用。此调用返回一个句柄，之后所有的使用都依靠这个句柄来标识。
- **epoll_ctl()** 系统调用。通过此调用向epoll对象中添加、删除、修改感兴趣的事件，返回0标识成功，返回-1表示失败。
- **epoll_wait()** 系统调用。通过此调用收集收集在epoll监控中已经发生的事件。

`int epoll_create(int size);` (旧写法, 旧接口, Linux 2.6.8 以前生效, 了解即可)

创建一个 epoll 对象, 返回该对象的描述符, 注意要使用 close 关闭该描述符。

`epoll_create()` 的 `size` 必须大于 0, 但现代 Linux 已忽略该值。实际工程通常使用: `epoll_create1(flags)`, 使用 0 或 `EPOLL_CLOEXEC`

如: `int epoll_fd = epoll_create1(0);`

使用 `epoll_create1(0)` 创建 **epoll** 实例。旧代码中可能使用 `epoll_create(size)`, 两者作用相同, 但现代 **Linux** 推荐 `epoll_create1()`。(Linux 2.6.27 引入)

int epoll_ctl(int epfd, int op, int fd, struct epoll_event *event);

操作控制 epoll 对象，主要涉及 epoll 红黑树上节点的一些操作，比如添加节点，删除节点，修改节点事件。

- **epfd**: 通过 epoll_create1 创建的 epoll 对象句柄（实例描述符）。
- **op**: 对红黑树的操作。
 - EPOLL_CTL_ADD: 注册新的fd到epfd中;
 - EPOLL_CTL_MOD: 修改已经注册的fd的监听事件
 - EPOLL_CTL_DEL: 从epfd中删除一个fd;
- **fd**: 需要添加监听的 socket 描述符，可以是监听套接字，也可以是与客户端通讯的通讯套接字。
- **event**: 需要监听的事件信息。

```
struct epoll_event {  
    __uint32_t events; /* 事件类型 */  
    epoll_data_t data; /* 与事件关联的  
    用户数据 */  
};
```

```
typedef union epoll_data {  
    void *ptr;  
    int fd;  
    uint32_t u32;  
    uint64_t u64;  
} epoll_data_t;
```

```
ev.events = EPOLLIN;  
ev.data.fd = client_fd;
```

配置	含义	本实训是否使用
EPOLLIN	fd 可读	使用
EPOLLOUT	fd 可写	暂不使用
不设置 EPOLLET	LT 水平触发	使用
设置 EPOLLET	ET 边沿触发	不使用

- events 可以是以下几个宏的集合：

类型	含义
EPOLLIN	表示对应的文件描述符可以读
EPOLLOUT	表示对应的文件描述符可以写
EPOLLERR	表示对应的文件描述符发生错误
EPOLLHUP	表示对应的文件描述符被挂断
EPOLLET	将EPOLL设为边缘触发(Edge Triggered)模式
EPOLLONESHOT	只监听一次事件，当监听结束之后，要继续监听的话，需要再次加入到EPOLL队列里

```
int epoll_wait(int epid, struct epoll_event *events, int maxevents,  
int timeout);
```

阻塞一段时间并等待事件发生，返回事件集合，也就是获取内核的事件通知。

- **epid**: `epoll_create` 返回的 `epoll` 对象描述符。
- **events**: 存放就绪的事件集合，这个是传出参数。
- **maxevents**: 代表可以存放的事件个数，也就是events数组的大小
- **timeout**: 阻塞等待的时间长短，以毫秒为单位，如果传入 -1 代表阻塞等待

• 返回值

返回值	含义
>0	代表有几个我们希望监听的事件发生了
=0	timeout 超时时间到了
<0	出错，可以通过 errno 值获取出错原因

举例：假设服务器通过调用 `accept` 函数成功与客户端建立连接并得到了通讯套接字 `connfd`,

- 如果需要关心这个客户端是否给服务器发送数据过来（读事件），我们需要将 `event` 参数的 `events` 成员的 `EPOLLIN` 置位为1
- 如果并不关心服务器是否可以往客户端写数据（写事件），我们可以将 `event` 参数的 `events` 成员的 `EPOLLOUT` 置位为 0

因此，可以对某个事件的关注通过0/1来设置，再通过 `epoll_ctl`函数来修改。这样当调用 `epoll_wait` 函数等待事件时，一旦发生了我们希望监听的事件时，`epoll_wait` 函数就会返回并通知我们哪个描述符发生了对应的事件。

问题：在epoll I/O多路复用中，某个socket读到一半，在这个socket上又有读事件来了怎么办？

为了避免在同一个socket上再次监听到同一个可读事件，可以在对应的描述符中添加 EPOLL_ONESHOT事件，其效果是监听到一次事件后就将对应的描述符从监听集合中移除，也就不会再被追踪到了。读操作完成后再把对应的文件描述符重新加入监听集合。

Epoll两种模式——LT与ET模式

- **LT模式（水平触发）——默认模式**

以读事件为例，当缓冲区有数据准备好的时候，此时会触发读事件，如果一直不读取缓冲区里的数据，epoll模型就会一直通知我们事件就绪，即epoll_wait中的events参数就会一直包含某个文件描述符的读事件。

- **event.events=EPOLLIN;**

Epoll两种模式——LT与ET模式

- **ET模式（边缘触发）**

ET模式与LT模式相反，当缓冲区就数据准备好的时候，也会触发读事件，但是只会触发一次，如果我们这次没有调用 `read/recv` 读取 或者 没有一次读完，后面就不会通知有读事件就绪了。简单来说，只有当缓冲区里的数据量发生变化的时候，才会通知我们一次，不会像LT模式那样一直通知。

- **`event.events=EPOLLIN|EPOLLET;`**

Epoll两种模式——LT与ET模式

LT（水平触发）模式，该模式epoll_wait返回可读事件，表明socket一定收到了数据，我们可以使用read函数来读取数据。如果指定读取的数据大于缓冲区数据，read函数不会阻塞，会返回实际读取到的数据大小。如果指定读取的数据小于缓冲区中数据，epoll_wait会继续被触发，因为还有读缓冲区中还有数据没有被读取完。在**ET（边缘触发）模式**下，只有新的数据到来时才会触发。如果指定读取的数据小于缓冲区中的数据，epoll_wait不会被继续触发

因此，使用ET模式时，有数据到来时，必须循环读取读缓冲区中的数据，直到read返回-1，并且errno错误码为EAGAIN，才算读取完了全部缓冲区中的内容

实训内容1：使用select函数实现服务器 (W2D4 必做)

实训内容 1：完成 SelectServer + TCP Client，实现基于 select 的多客户端文本通信。

- ① 服务端通过命令行接收 IP 和端口，例如：./SelectServer 127.0.0.1 8888；仅使用 select 及相关 API，不使用多线程、多进程和线程池。
- ② 服务端需要实时输出：当前连接数量、客户端 IP、客户端端口、客户端连接/断开事件，以及收到的文本消息。
- ③ TCP 消息以 \n 作为一条完整消息的结束标志。服务端需要维护每个客户端的接收缓冲区，按 \n 分帧处理消息。
- ④ 当服务端收到客户端发送的 quit\n 时，视为客户端主动退出；当 recv() == 0 时，表示客户端已关闭连接。
- ⑤ 出现 quit\n、recv() == 0 或通信错误时，服务端应执行：
FD_CLR(client_fd) -> close(client_fd) -> 更新当前连接数量。
- ⑥ 使用至少 3 个客户端连接同一服务端进行测试：一个客户端退出后，其余客户端仍能继续发送和接收消息。

验收项目	现场检查方法	通过标准
编译与启动	学生独立编译并启动服务端和客户端	无编译错误，服务端成功监听指定 IP 和端口
3个客户端连接	同时启动至少3个客户端	3个客户端均连接成功，服务端不中断
连接信息输出	观察服务端终端	显示连接数、客户端 IP、端口及连接事件
文本消息接收	3个客户端分别发送消息	服务端能显示各客户端发送的完整文本
<code>\n</code> 消息分帧	测试半条消息和一次发送多条消息	能按 <code>\n</code> 提取完整消息，不把一次 <code>recv()</code> 当成一条消息
<code>quit\n</code> 退出	一个客户端发送 <code>quit\n</code>	服务端删除并关闭该客户端 fd，连接数正确减少
断开连接处理	直接关闭一个客户端	服务端识别 <code>recv() == 0</code> ，执行 <code>FD_CLR + close</code>
其他客户端不受影响	一个客户端退出后，其余客户端继续发送	剩余客户端仍能正常通信
<code>select</code> 代码检查	学生现场指出关键代码	正确使用 <code>FD_ZERO</code> 、 <code>FD_SET</code> 、 <code>select</code> 、 <code>FD_ISSET</code> 、 <code>FD_CLR</code>

实训内容1：使用 `select` 函数实现服务器的验收标准

实训内容2：使用 epoll 模型实现 TCP 服务器 EpollServer (W2D5 必做)

- ① EpollServer 运行时通过命令行接收 IP 和端口，例如：./EpollServer 127.0.0.1 8888。
- ② 使用 epoll 相关 API，采用 LT + EPOLLIN 模式；不得使用 select、多线程、多进程或线程池。
- ③ 服务端实时输出客户端连接数、客户端 IP 和端口、连接/断开事件及收到的消息。
- ④ 实现配套 TCP Client，消息以 \n 分帧，输入 quit 时退出并清理连接。
- ⑤ 最终展示3个客户端同时连接，且任一客户端退出后，其他客户端仍可正常通信。

实训内容2：使用 epoll 函数实现服务器验收标准

验收项目	现场检查方法	通过标准
编译与启动	学生独立编译并启动 EpollServer 和 TCP Client	无编译错误，服务端成功监听指定 IP 和端口
3个客户端通信	同时连接3个客户端并分别发送消息	3个客户端连接成功；服务端显示连接数、IP、端口和完整消息
\n 消息分帧	测试半条消息和一次发送多条消息	能按 \n 拼接和提取消息，不把一次 recv() 当成一条消息
连接退出与清理	一个客户端发送 quit 或直接关闭，其他客户端继续发送	执行 EPOLL_CTL_DEL + close，连接数减少，剩余客户端通信正常
epoll代码检查	学生现场指出事件注册、等待和删除代码	正确使用 epoll_create1()、epoll_ctl()、epoll_wait() 及 LT + EPOLLIN；未使用 select、多线程或多进程

\n 分帧可以直接运行自动测试，不必全部手工操作，可提前运行自动测试并展示结果

实训内容3：对select服务器和epoll服务器进行对比测试 (选做)

• 实训内容3.1 连接建立速度测试

某个时刻连续向server发起大量连接请求，比较两种I/O模型（select和epoll）下Server端的连接接收速度。在客户端记录下连接数完全建立后所花费的时间。

• I.启动服务端程序.

Selectserver命令: (./SelectServer 192.168.0.30 8000)

EpollServer命令: (./EpollServer 192.168.0.30 8000)

参数1(192.168.0.30)为server绑定的IP, 参数2(8000)为server监听的端口号; 实际使用IP和端口号自定

实训内容3：对select服务器和epoll服务器进行对比测试 (选做)

实训内容3.1 连接建立速度测试

编写客户端程序，该客户端程序可以接收连接数量作为参数

- II.启动客户端程序

命令: `./testlink 192.168.0.30 8000 800`

参数1(192.168.0.30)是server端的IP, 参数2(8000)是server监听的端口, 参数3(800)是想要建立连接的数量, 等连接全部建立完毕后程序会自动打印出所花费的时间及成功建立的连接数.

- 每种连接数量记录5组数据, 去除一个最大及最小值后, 取余下的3组数据的平均值作为最终结果。

实训内容3：对select服务器和epoll服务器进行对比测试 (选做)

实训内容3.2 数据传输性能测试

客户端创建若干线程，每个线程与服务器建立一个连接。连接建立后向服务器发送读取数据请求，然后读取server端返回的数据.如此反复循环。

- 每个 client请求server返回的数据字节数为1K(1024bytes)大小。当连接全部建立后，记录此时的服务程序对应的CPU占用率及内存使用率。每个连接数量记录下12组数据供分析使用。
- 分析结果中将除去一个最大值及最小值，取余下的10组数据的平均值作为最终结果。

实训内容3.2 数据传输性能测试

- I. 启动服务端

Selectserver命令: (./SelectServer 192.168.0.30 8000)

EpollServer命令: (./EpollServer 192.168.0.30 8000)

参数1(192.168.0.30)为server绑定的IP, 参数2(8000)为server监听的端口号;实际使用IP和端口号自定

实训内容3：对select服务器和epoll服务器进行对比测试 (选做)

- II.启动客户端

命令: ./activelink 192.168.0.30 8000 800

参数1(192.168.0.30)是server端的IP, 参数2(8000)是server监听的端口,参数3(800)是你想要建立的线程数(连接数)。因为每个线程建立一个连接, 所以此数量亦即建立的连接数。

- III. 查看连接数量

netstat -la | grep "192.168.0.250" | wc -l 查看连接数量,等待建立完成。此处192.168.0.250为客户端机器IP地址

- IV.记录。连接全部建立后等待5-6分钟, 待系统稳定下来后top查看并记录12 组Server 程序所占CPU/内存使用率。

实训内容3：对select服务器和epoll服务器进行对比测试 (选做) (高并发区间仅测 epoll)

连接建立速度测试表格

链接数量	Select服务器	Epoll服务器
100		
200		
500		
800		
1000		
3000	/	
5000	/	
8000	/	
10000	/	
12000	/	

实训内容3：对select服务器和epoll服务器进行对比测试 (选做)

连接建立速度测试表格

链接数量	Select服务器	Epoll服务器
100		
200		
500		
800		
1000		
3000	/	
5000	/	
8000	/	
10000	/	
12000	/	

实训内容3：对select服务器和epoll服务器进行对比测试 (选做)

• 数据传输性能测试表格

链接数量	Select服务器 [cpu%, mem%]	Epoll服务器[cpu%, mem%]
100		
200		
500		
800		
1000		
3000	/	
5000	/	
8000	/	
10000	/	
12000	/	

Webserver V1.0版 (W2D5必做, 完成 V1.0 的 epoll WebServer)

epoll 版高并发 Web Server



mini_webserver

工程基础模块 Day1 && Day4

- 构建子模块 D1
Makefile
编译、清理、调试版本
- 配置子模块 D1
Config.c
端口、根目录、线程数
- 日志子模块 D1,D4
log.c
访问日志、错误日志

用户数据模块 Day2 && Day3

- 用户结构子模块 D2-D3
user.c
链表、二叉树、增删改查
- CSV存储子模块 D2
csv_store.c
用户数据加载和保存
- 索引查找子模块 D3
user_index.c
按用户名快速查找

网络服务模块 Day6 && Day7

- TCP监听子模块 D6
server.c
socket, bind, listen
- 连接管理子模块 D7
client.c
连接与状态、关闭释放

并发处理模块 Day4-5 && Day8-9

- 进程模型子模块 D4
process.c
父子进程协同
- 线程同步子模块 D5
thread.c
锁、条件变量、共享资源保护
- 线程池子模块 D8
thread_pool.c
任务队列、工作线程
- 事件驱动子模块 D9
event_loop.c
select / epoll

HTTP业务模块 Day10-Day14

- 请求解析子模块 D10
http_request.c
方法、URL、Header、Body
- 静态资源子模块 D11
static_file.c
HTML、CSS、图片读取
- GET/POST处理子模块 D12
handler.c
表单提交、参数读取
- 路由配置子模块 D13
router.c
URL匹配、处理函数绑定
- 认证安全子模块 D14
auth.c
登录校验、权限控制、路径检查
- 响应生成子模块 D10-D14
http_response.c
状态码、响应头、响应体

测试模块 Day1-15

- 每日增加功能的相关测试
如: test_day01.sh
- 如: bench.c 并发压力测试/响应时间等

一个具备高并发连接管理基础的webserver服务器

Select用做服务器技术时候的问题:

- select需要反复拷贝fd_set
- select返回后还要遍历所有fd
- FD_SETSIZE通常有限制
- 连接多、活跃少时浪费明显

poll 的改进与局限:

解决了 select 的 FD_SETSIZE 固定限制, 但每轮仍需传递并遍历完整 fd 数组。
因此本实训直接使用 epoll 实现 V1.0 WebServer。



一个具备高并发连接管理基础的webserver服务器

使用Epoll LT 模式的 webserver服务器

- `event.events = EPOLLIN`

新增文件包括:

- `include/epoll_server.h`
- `src/epoll_server.c`
- `tests/test_day10.sh`

修改文件包括

- `src/main.c`
- `Makefile`
- `README.md`
- `docs/debug-log.txt` (可选)



V1.0版 mini_webserver-功能要求

- 支持 serve-epoll <max_requests>
- 使用 epoll_create
- 使用 epoll_ctl添加监听 socket
- 新客户端连接后，把client_fd添加到epoll客户端
- 请求到来后，读取 HTTP 请求并返回响应
- 支持 /hello和/users/zhangsan
- 未知路径返回404 Not Found
- 客户端关闭或出错时，从 epoll删除fd并关闭fd主任务
- 使用 LT 模式
- ET模式和EPOLLONESHOT作为扩展



V1.0 版 mini_webserver 最低验收标准

验收项目	现场检查方法	通过标准
编译与启动	学生独立编译并执行 <code>serve-epoll</code> <code><max_requests></code>	无编译错误，服务器成功监听指定端口
<code>/hello</code> 路由	执行 <code>curl -i</code> <code>http://127.0.0.1:8080/hello</code>	返回 <code>200 OK</code> 和正确响应正文
用户路由	执行 <code>curl -i</code> <code>http://127.0.0.1:8080/users/zhangsan</code>	返回 <code>200 OK</code> 和张三的用户信息
404处理	执行 <code>curl -i</code> <code>http://127.0.0.1:8080/missing</code>	返回 <code>404 Not Found</code>
多请求处理	同时或连续发送多个 HTTP 请求	每个请求均能获得完整 HTTP 响应，服务器不中断
请求数量控制	请求总数达到 <code>max_requests</code>	服务器完成当前请求后正常退出
连接清理	观察响应完成和客户端断开后的处理	执行 <code>EPOLL_CTL_DEL + close(client_fd)</code> ，无连接泄漏
epoll代码检查	学生现场指出事件循环关键代码	使用 <code>epoll_create1()</code> 、 <code>epoll_ctl()</code> 、 <code>epoll_wait()</code> 、LT + <code>EPOLLIN</code>
技术限制检查	检查服务端源代码	未使用 <code>select</code> 、多线程、多进程或线程池
自动测试	运行项目测试脚本	两个路由、404、请求计数和关闭清理全部通过

对比项	实训内容2: EpollServer	WebServer V1.0
实训重点	掌握 epoll 事件循环和 fd 生命周期	将 epoll 应用于 HTTP Web Server
通信协议	自定义 TCP 文本协议	HTTP 协议
测试客户端	配套 TCP Client	<code>curl</code> 或浏览器
消息边界	使用 <code>\n</code> 分帧	使用 <code>\r\n\r\n</code> 判断 HTTP 请求头结束
收到数据后	显示或转发文本消息	解析请求方法、路径和 HTTP 版本
退出规则	<code>quit\n</code> 、 <code>recv()==0</code> 或错误	响应完成、 <code>recv()==0</code> 或错误
服务端响应	文本消息或广播	HTTP 状态行、响应头和响应正文
业务功能	多客户端文本通信	规定路由、页面响应和 <code>404 Not Found</code>
测试证据	3个 TCP Client 同时通信	<code>curl -i</code> 检查路由、状态码和响应头

- 实训内容2的 EpollServer: 重点练习如何用 epoll 管理多个 TCP 连接。
- WebServer V1.0: 在相同的 epoll 连接管理框架上, 增加 HTTP 协议解析、路由和响应。
- 两者 “基础事件服务器” 到 “HTTP 应用服务器” 的递进关系。

两者共同使用的核心流程是:

```
socket -> bind -> listen
      -> epoll_create1
      -> EPOLL_CTL_ADD listen_fd
      -> epoll_wait
      -> accept
      -> EPOLL_CTL_ADD client_fd
      -> recv
      -> 处理协议
      -> EPOLL_CTL_DEL
      -> close
```



真正发生变化的是“处理协议”部分：

实训内容2:

recv

- > 追加到客户端缓冲区
- > 按 \n 提取文本
- > 判断 quit
- > 显示或广播消息

WebServer V1.0:

recv

- > 追加到客户端缓冲区
- > 判断是否收到完整 HTTP 请求
- > 解析方法和路径
- > 匹配路由
- > 返回 200 或 404
- > 关闭连接

代码迁移关系可以表示为:

EpollServer

- epoll 事件循环 保留
- listen_fd/client_fd 管理 保留
- accept/recv 保留
- EPOLL_CTL_DEL + close 保留
- \n 文本处理 替换为 HTTP 解析和响应



建议学生按以下顺序完成：

1. 先完成实训内容2，确保3个客户端连接、消息分帧和退出清理正确。
2. 复制或复用 **EpollServer** 的事件循环。
3. 将文本消息处理函数替换为 **HTTP** 请求处理函数。
4. 增加课件规定的两个路由、404 响应和关闭清理。
5. 使用 `curl -i` 验证状态码、响应头和正文。

总结：实训内容2解决“怎样用 **epoll** 管理多个 **TCP** 连接”，**WebServer V1.0**解决“连接就绪后怎样正确处理 **HTTP** 请求”

